OAuth 2.0, OpenID Connect and Google Login with Spring Boo

1. Moving Beyond Our Own Authentication System

Until now, our application controlled the complete security flow:

```
User sends username and password
            ↓
Spring Boot authenticates the user
            ↓
Application generates a JWT
            ↓
Client sends the JWT with future requests
```

Our API validates the token's signature and trusts the claims only when the signature is valid.

JWT solves an important problem:

> How can an API verify the information inside a token without trusting the client?

Now consider a different requirement:

```
Continue with Google
```

We cannot ask the user to send their Google password to our application:

```
POST /login-with-google

email=student@gmail.com
googlePassword=???
```

Our application should never receive, process or store the user's Google password.

---

2. The Delegated Access Problem

Suppose we build a productivity application and add this feature:

> Connect your Google Calendar.

Our application needs permission to read the user's calendar. A dangerous design would be:

```
User gives Google credentials to our application
                    ↓
Our application logs in to Google as the user
                    ↓
Our application accesses Google Calendar
```

This creates two major problems:

1. The user must share their Google password with another application.
2. The application receives far more access than it needs.

The same Google credentials could potentially expose Gmail, Drive, Photos, Calendar, YouTube and other services, even though the application only needs calendar access.

OAuth 2.0 solves this by replacing shared credentials with delegated, limited access:

```
User signs in directly with Google
                ↓
User allows the application to read the calendar
                ↓
Google gives the application an access token
                ↓
The application uses the token with Google Calendar API
```

> OAuth 2.0 is primarily a framework for delegated authorization.

Authorization answers:

```
What are you allowed to do?
```

3. The Four OAuth Roles

Role	Example	Responsibility
Resource Owner	User	Owns or controls access to the protected resource
Client	CoderArmy application	Requests delegated access on behalf of the user
Authorization Server	Google's authorization system	Authenticates the user, obtains consent and issues tokens
Resource Server	Google Calendar API	Hosts the protected resource and validates access tokens

Resource Owner

The resource owner is usually the user who controls the protected data.

Client

In OAuth terminology, **client does not necessarily mean browser**. It means the application requesting access.

A client can be a:

- Spring Boot application
- Mobile application
- Desktop application
- Browser-based application

Authorization Server

The authorization server performs tasks such as:

```
Authenticate the user
        ↓
```

```
Ask for authorization or consent
        ↓
Issue tokens
```

Resource Server

The resource server contains the protected resource. For example, Google Calendar API receives an access token with the request:

```
Authorization: Bearer <access-token>
```

It validates the token and decides whether the requested operation is allowed.

---

4. Scope and Consent

The authorization server must know exactly what the client wants to access. OAuth represents this requested access through **scopes**.

Conceptually, a calendar application could request:

```
calendar.read
```

instead of requesting unrestricted access to everything in the user's Google account.

> A scope represents the access being requested or granted.

Google can then display a consent screen:

```
CoderArmy App wants permission to:

✓ View your basic profile
✓ Read your calendar
```

The user can approve or reject the request. This is the consent step.

```
Client requests specific scopes
              ↓
Authorization server displays consent
              ↓
User approves or rejects the requested access
```

---

5. OAuth and OIDC Tokens

Access Token

After authorization, the client needs a credential it can present to the resource server. That credential is the **access token**.

```
Authorization Server
        ↓ access token
Client
        ↓ bearer token
Resource Server
```

> An access token is a credential used by a client to access a protected resource.

An access token may be a JWT, but OAuth 2.0 does not require it to be one. It can also be an opaque, random-looking value.

The client should generally treat an access token as an opaque credential unless the authorization server explicitly documents its format.

The Authentication Problem

OAuth solves delegated authorization:

```
What can this client access?
```

However, login requires an identity answer:

```
Who is the user who signed in?
```

OAuth 2.0 alone is not an identity protocol. **OpenID Connect**, commonly called OIDC, adds authentication and identity concepts on top of OAuth 2.0.

Protocol	Primary purpose	Main question
OAuth 2.0	Delegated authorization	What can this application access?
OpenID Connect	Authentication and identity	Who signed in?

ID Token

OIDC introduces the **ID token**. It is intended for the client and contains information about the authentication event and the user's identity.

An ID token is normally a signed JWT and may contain claims such as:

```
{
  "sub": "123456789",
  "iss": "https://accounts.google.com",
  "aud": "our-google-client-id",
  "name": "Aditya",
  "email": "adi@gmail.com"
}
```

Important claims include:

Claim	Meaning
`sub`	Stable subject identifier for the user at that provider
`iss`	Issuer that created the token
`aud`	Client for which the token was issued
`exp`	Expiration time

Refresh Token

Access tokens should expire. A refresh token may be issued so the client can obtain a new access token without forcing the user to repeat the complete login and consent flow.

```
Access token expires
        ↓
Client sends refresh token to authorization server
        ↓
Authorization server issues a new access token
```

A refresh token is sent to the authorization server's token endpoint, not to a normal resource API.

Token Comparison

Token	Used by	Sent to	Purpose
Access token	Client	Resource server	Access a protected API
ID token	Client	Client application	Verify authentication and obtain identity information
Refresh token	Client	Authorization server	Obtain a new access token

> An ID token is not a replacement for an access token. Do not use an ID token to call a protected API unless that API explicitly supports it.

---

6. Authorization Code Flow

The client must obtain tokens without casually exposing them through browser redirects. The Authorization Code Flow handles this through two stages:

1. The browser receives a short-lived authorization code.
2. The backend exchanges that code for tokens through a direct server-to-server request.

Step 1: Start the Authorization Request

The user visits the application and clicks:

```
Continue with Google
```

The application redirects the browser to Google's authorization endpoint.

The authorization request contains values such as:

```
client_id
redirect_uri
scope
response_type=code
state
```

`client_id`

Google gives the application a client ID when it is registered.

```
client_id = coderarmy-app-123
```

It identifies which application is making the request. A client ID is an identifier, not a secret.

`redirect_uri`

This is where Google sends the browser after completing the authentication and authorization process.

For the local Spring Boot application:

```
http://localhost:8080/login/oauth2/code/google
```

The redirect URI must exactly match an authorized URI registered with Google.

`scope`

For basic OIDC login, the application requests:

```
openid
profile
email
```

The `openid` scope tells the provider that this is an OpenID Connect request.

`response_type=code`

This tells the authorization server to return an authorization code rather than placing the final tokens directly in the browser redirect.

`state`

The client generates a random state value before the redirect:

```
Client creates state
        ↓
State travels with authorization request
        ↓
Authorization server returns the same state
        ↓
Client verifies it
```

The state value correlates the callback with the original request and helps protect the authorization flow against attacks such as login CSRF.

Step 2: Google Authenticates the User

The browser is now on Google's domain. Google can authenticate the user with a password, passkey, two-factor authentication or another supported mechanism.

The important security boundary is:

```
Google credentials go from the browser to Google.
They never go to our Spring Boot application.
```

Step 3: The User Gives Consent

Google displays the permissions requested by the application. For a basic login, these may include profile and email information. For a Calendar integration, the screen could also request calendar access.

After approval, Google knows that:

```
The user authenticated successfully
                    +
The user approved the requested access
```

Step 4: Google Returns an Authorization Code

Google redirects the browser to the registered callback:

```
http://localhost:8080/login/oauth2/code/google
    ?code=A7D91XYZ
    &state=abc123
```

The authorization code is a short-lived, temporary credential representing the successful authorization step.

```
Authorization code ≠ Access token
```

The code is not normally used to call a resource API. Its purpose is to be exchanged.

Step 5: The Backend Exchanges the Code for Tokens

The Spring Boot backend contacts Google's token endpoint:

```
Spring Boot backend
    ├── authorization code
    ├── redirect URI
    └── client authentication and/or PKCE proof
                ↓
Google token endpoint
                ↓
Access token + ID token + optional refresh token
```

Front Channel vs Back Channel

Channel	Communication path	Typical data
Front channel	Through the user's browser	Authorization request, code and state
Back channel	Directly between backend and token endpoint	Code exchange and tokens

The authorization code passes through the browser, but the final tokens are obtained through the back channel.

---

7. PKCE from First Principles

An attacker who steals an authorization code may try to exchange it before the legitimate client. PKCE binds the authorization request to the client instance that later performs the token exchange.

PKCE stands for:

```
Proof Key for Code Exchange
```

It is commonly pronounced **pixy**.

Creating the Proof

Before starting authorization, the client generates a long, random value:

```
code_verifier = a-long-random-secret
```

It hashes the verifier, normally using SHA-256:

```
SHA-256(code_verifier)
          ↓
code_challenge
```

During the Authorization Request

The client sends the `code_challenge` to the authorization server but keeps the `code_verifier` locally.

```
Client ── code_challenge ──→ Authorization server
```

The authorization server remembers the challenge associated with that request.

During the Token Exchange

The client later sends:

```
authorization_code
code_verifier
```

The authorization server hashes the received verifier and compares the result with the original challenge:

```
SHA-256(received code_verifier)
                ↓
        calculated challenge
                ↓
calculated challenge == original code_challenge?
```

If they match, the server can issue tokens. If they do not match, the request is rejected.

An attacker may steal the authorization code, but without the verifier the stolen code cannot be successfully exchanged.

> A client secret authenticates a confidential client application. PKCE binds the code exchange to the client instance that initiated the authorization request. They solve related but different problems.

---

8. Registering the Application with Google

Google must know which application is requesting login and which callback addresses it can trust.

```
Register application with Google
            ↓
```

```
Receive Client ID and Client Secret
            ↓
Register allowed redirect URI
```

For this local demo, the authorized redirect URI is:

```
http://localhost:8080/login/oauth2/code/google
```

Spring Security uses this default redirect URI template:

```
{baseUrl}/login/oauth2/code/{registrationId}
```

Here:

```
baseUrl       = http://localhost:8080
registrationId = google
```

---

9. Spring Boot Application Flow

The application uses Google for authentication but maintains its own internal user record:

```
Google authenticates the user
            ↓
Spring receives an OidcUser
            ↓
Look up provider + provider subject
       ┌────┴────┐
   Not found    Found
       ↓          ↓
    Insert    Update/load
       └────┬────┘
            ↓
Internal AppUser
            ↓
Spring Security session
```

Google owns the external identity. Our database owns application-specific information such as:

- Internal user ID
- Application role
- Account status
- Preferences
- Subscription or membership data

---

10. Project Structure

```
oauth2-demo
├── pom.xml
└── src
    └── main
        ├── java
        │   └── in.coderarmy.oauth2demo
        │       ├── OAuth2DemoApplication.java
        │       ├── config
        │       │   └── SecurityConfig.java
        │       ├── controller
        │       │   └── UserController.java
        │       ├── entity
        │       │   └── AppUser.java
        │       ├── repository
        │       │   └── AppUserRepository.java
        │       └── service
        │           ├── AppUserService.java
        │           └── CustomOidcUserService.java
        └── resources
            └── application.properties
```

---

11. Required Dependencies

The OAuth/OIDC client starter is:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-client</artifactId
```

```
>
</dependency>
```

The complete demo also needs Spring Web, Spring Data JPA and a database driver. For a minimal in-memory demo, the relevant dependencies are:

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-oauth2-client</artifa
ctId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>
</dependencies>
```

The OAuth2 client starter provides the Spring Security support required for OAuth2 Login. Adding `spring-boot-starter-security` separately is optional when the OAuth2 client starter is already present.

Why is our application called an OAuth client?

```
Spring Boot application
        ↓
Requests authentication and delegated access from Google
```

12. User Entity

```java
package in.coderarmy.oauth2demo.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class AppUser {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;

    private String provider;

    @Column(name = "provider_subject")
    private String providerSubject;

    private String role;

    public AppUser() {
    }
```

```
    public AppUser(
            String name,
            String email,
            String provider,
            String providerSubject,
            String role
    ) {
        this.name = name;
        this.email = email;
        this.provider = provider;
        this.providerSubject = providerSubject;
        this.role = role;
    }

    // Generate getters and setters.
}
```

There is deliberately no password field. Google owns and verifies the Google password; our application never receives it.

Why Store `providerSubject`?

Google includes a `sub` claim in the ID token:

```
{
  "sub": "109324823948234",
  "email": "student@gmail.com",
  "name": "Rohit"
}
```

`sub` means **subject**. It is the provider's stable identifier for the user.

We therefore store:

```
provider = google
```

```
providerSubject = value of sub
```

The combination of provider and subject is a better external identity key than email because an email address may be changed, missing or unverified depending on the provider and scopes.

In a real database, this pair should normally be unique.

13. Repository

```java
package in.coderarmy.oauth2demo.repository;

import in.coderarmy.oauth2demo.entity.AppUser;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface AppUserRepository
        extends JpaRepository<AppUser, Long> {

    Optional<AppUser> findByProviderAndProviderSubject(
            String provider,
            String providerSubject
    );
}
```

This method asks:

```
Does our database already contain a user with:

provider = google
providerSubject = 123456
```

14. Application User Service

The signup operation happens inside our application after Google has authenticated the user.

```
package in.coderarmy.oauth2demo.service;

import in.coderarmy.oauth2demo.entity.AppUser;
import in.coderarmy.oauth2demo.repository.AppUserRepository;
import org.springframework.security.oauth2.core.oidc.user.Oid
cUser;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transaction
al;

import java.util.Optional;

@Service
public class AppUserService {

    private final AppUserRepository appUserRepository;

    public AppUserService(
            AppUserRepository appUserRepository
    ) {
        this.appUserRepository = appUserRepository;
    }

    @Transactional
    public AppUser registerOrUpdate(
            String provider,
            OidcUser oidcUser
    ) {
        String providerSubject = oidcUser.getSubject();
        String name = oidcUser.getClaimAsString("name");
        String email = oidcUser.getClaimAsString("email");

        Optional<AppUser> existingUser =
```

```
                appUserRepository
                        .findByProviderAndProviderSubject(
                                provider,
                                providerSubject
                        );

        if (existingUser.isPresent()) {
            AppUser user = existingUser.get();
            user.setName(name);
            user.setEmail(email);

            // The managed entity is updated through JPA dirt
y checking.
            return user;
        }

        AppUser newUser = new AppUser(
                name,
                email,
                provider,
                providerSubject,
                "USER"
        );

        return appUserRepository.save(newUser);
    }

    public Optional<AppUser> findByProviderAndSubject(
            String provider,
            String subject
    ) {
        return appUserRepository
                .findByProviderAndProviderSubject(
                        provider,
                        subject
                );
```

```
    }
}
```

The database flow is:

```
SELECT *
FROM app_user
WHERE provider = 'google'
  AND provider_subject = '123';
```

- If the user exists, update the latest profile details and load the internal account.
- If the user does not exist, insert a new `AppUser` with the default `USER` role.

This is the application's OAuth/OIDC signup process.

---

15. Custom OIDC User Service

Spring Security already knows how to communicate with an OIDC provider. We should preserve that built-in processing and add only our application-specific database logic.

```
Spring performs normal OIDC processing
                ↓
Spring returns an OidcUser
                ↓
Our code saves or updates the internal user
                ↓
Return the OidcUser to Spring Security
```

```
package in.coderarmy.oauth2demo.service;

import org.springframework.security.oauth2.client.oidc.userin
fo.OidcUserRequest;
import org.springframework.security.oauth2.client.oidc.userin
fo.OidcUserService;
```

```
import org.springframework.security.oauth2.client.userinfo.OA
uth2UserService;
import org.springframework.security.oauth2.core.OAuth2Authent
icationException;
import org.springframework.security.oauth2.core.oidc.user.Oid
cUser;
import org.springframework.stereotype.Service;

@Service
public class CustomOidcUserService implements
        OAuth2UserService<OidcUserRequest, OidcUser> {

    private final OidcUserService delegate;
    private final AppUserService appUserService;

    public CustomOidcUserService(
            AppUserService appUserService
    ) {
        this.appUserService = appUserService;
        this.delegate = new OidcUserService();
    }

    @Override
    public OidcUser loadUser(
            OidcUserRequest userRequest
    ) throws OAuth2AuthenticationException {

        // Let Spring perform the provider interaction and OI
DC processing.
        OidcUser oidcUser = delegate.loadUser(userRequest);

        String provider = userRequest
                .getClientRegistration()
                .getRegistrationId();

        // Run our application-specific signup/update logic.
```

```
            appUserService.registerOrUpdate(provider, oidcUser);

            return oidcUser;
        }
    }
```

The call below represents significant built-in processing:

```
OidcUser oidcUser = delegate.loadUser(userRequest);
```

Conceptually, by this stage:

```
Authorization code has been exchanged
                ↓
Access token and ID token are available
                ↓
Spring validates and processes OIDC information
                ↓
Spring may call the UserInfo endpoint
                ↓
Spring creates an OidcUser
```

---

16. Security Configuration

```
package in.coderarmy.oauth2demo.config;

import in.coderarmy.oauth2demo.service.CustomOidcUserService;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.bui
lders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {
```

```
    @Bean
    public SecurityFilterChain securityFilterChain(
            HttpSecurity http,
            CustomOidcUserService customOidcUserService
    ) throws Exception {

        http
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/").permitAll()
                        .anyRequest().authenticated()
                )
                .oauth2Login(oauth -> oauth
                        .userInfoEndpoint(userInfo -> userInf
o
                                .oidcUserService(customOidcUs
erService)
                        )
                        .defaultSuccessUrl("/profile", true)
                );

        return http.build();
    }
}
```

The `oauth2Login()` configuration enables the OAuth 2.0 Login and OIDC flow. Spring Security manages the protocol endpoints involved in:

```
Authorization endpoint → redirect user to Google
Token endpoint         → exchange code for tokens
Redirection endpoint   → /login/oauth2/code/google
UserInfo endpoint      → obtain additional profile claims when required
```

17. Controller

```
package in.coderarmy.oauth2demo.controller;

import in.coderarmy.oauth2demo.entity.AppUser;
import in.coderarmy.oauth2demo.service.AppUserService;
import org.springframework.security.core.annotation.Authentic
ationPrincipal;
import org.springframework.security.oauth2.core.oidc.user.Oid
cUser;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestControlle
r;

import java.util.LinkedHashMap;
import java.util.Map;

@RestController
public class UserController {

    private final AppUserService appUserService;

    public UserController(
            AppUserService appUserService
    ) {
        this.appUserService = appUserService;
    }

    @GetMapping("/")
    public String home() {
        return """
                Public Home

                Login using:
                http://localhost:8080/oauth2/authorization/go
ogle

                """;
```

```
    }

    @GetMapping("/profile")
    public Map<String, Object> profile(
            @AuthenticationPrincipal OidcUser oidcUser
    ) {
        AppUser appUser = appUserService
                .findByProviderAndSubject(
                        "google",
                        oidcUser.getSubject()
                )
                .orElseThrow();

        Map<String, Object> response = new LinkedHashMap<>();

        response.put("internalUserId", appUser.getId());
        response.put("provider", appUser.getProvider());
        response.put("subject", oidcUser.getSubject());
        response.put("name", oidcUser.getClaimAsString("nam
e"));
        response.put("email", oidcUser.getClaimAsString("emai
l"));
        response.put("role", appUser.getRole());

        return response;
    }
}
```

After login, the response may look like:

```
{
  "internalUserId": 1,
  "provider": "google",
  "subject": "108293472983749823749",
  "name": "Rohit Negi",
  "email": "example@gmail.com",
```

```
  "role": "USER"
}
```

The data comes from two different systems:

Data	Source
`subject`, `name`, `email`	Google/OIDC claims
`internalUserId`, `role`	Our application database

18. Client Registration Properties

```
spring.application.name=oauth2-demo

# Google OAuth 2.0 / OIDC
spring.security.oauth2.client.registration.google.client-id=
${GOOGLE_CLIENT_ID}
spring.security.oauth2.client.registration.google.client-secr
et=${GOOGLE_CLIENT_SECRET}
spring.security.oauth2.client.registration.google.scope=openi
d,profile,email

# JPA demo configuration
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
```

Because Google is a provider recognized by Spring Boot, the normal authorization, token and UserInfo endpoint URLs do not need to be configured manually.

Do Not Hardcode the Client Secret

Keep the property placeholders in `application.properties` and supply the actual values through environment variables:

```
GOOGLE_CLIENT_ID
```

```
GOOGLE_CLIENT_SECRET
```

In IntelliJ IDEA:

```
Run
  → Edit Configurations
  → Select the Spring Boot application
  → Environment variables
```

Add both variables there. Spring resolves them at startup:

```
Environment variable
        ↓
${GOOGLE_CLIENT_ID}
        ↓
Spring client registration
```

Never commit a real client secret to Git or show it in a public recording.

---

19. Google Cloud Console Setup

Step 1: Create a Google Cloud Project

Open Google Cloud Console and create a project, for example:

```
spring-oauth-demo
```

Step 2: Open Google Auth Platform

Open **Google Auth Platform** for the project. If the project has not been configured before, select **Get Started**.

Step 3: Configure Branding

Enter the application details:

```
App name: CoderArmy OAuth Demo
```

```
User support email: your-email@gmail.com
```

Also provide the required developer contact information.

Step 4: Select the Audience

For a demo that should work with normal Gmail accounts, choose:

```
External
```

If the application remains in testing mode, add the Google accounts that will test the login flow as test users.

Step 5: Configure Data Access and Scopes

For basic OIDC login, request only:

```
openid
profile
email
```

Follow the principle of least privilege: request only the scopes the application genuinely needs.

Step 6: Create an OAuth Client

Open:

```
Google Auth Platform
  → Clients
  → Create Client
```

Choose:

```
Application type: Web application
```

Step 7: Authorized JavaScript Origins

This server-side Spring Boot demo does not use Google's browser-side JavaScript SDK, so **Authorized JavaScript origins** can remain empty.

Step 8: Add the Authorized Redirect URI

Add this exact URI:

```
http://localhost:8080/login/oauth2/code/google
```

The scheme, host, port and path must match. Even a small mismatch can cause a `redirect_uri_mismatch` error.

Step 9: Create the Client

Google generates:

```
Client ID
Client Secret
```

Store both as environment variables for the Spring Boot application.

---

20. Running and Testing the Application

Start the Spring Boot application and open:

```
http://localhost:8080/
```

To start Google login directly, visit:

```
http://localhost:8080/oauth2/authorization/google
```

The endpoint name contains the registration ID from the properties:

```
google
```

What Spring Does with This Request

```
GET /oauth2/authorization/google
                ↓
Spring finds ClientRegistration "google"
                ↓
Spring constructs the authorization request
                ↓
Spring sends a 302 redirect
                ↓
Browser opens Google's authorization endpoint
```

The generated request includes values such as:

```
client_id
redirect_uri
scope
state
response_type=code
```

Google Authentication

The user enters credentials on Google's domain:

```
Browser → Google
```

The credentials are not submitted to `localhost:8080`.

Callback and Code Exchange

After successful authentication and consent, Google redirects the browser to:

```
http://localhost:8080/login/oauth2/code/google
```

The callback contains a code and the returned state value:

```
/login/oauth2/code/google?code=ABC...&state=XYZ...
```

Spring validates the response and performs the back-channel code exchange with Google's token endpoint. It then processes the OIDC result and creates an `OidcUser`.

The `CustomOidcUserService` saves or updates the corresponding internal `AppUser`, and the success handler redirects to:

```
/profile
```

21. What Authenticates Later `/profile` Requests?

In this traditional server-side OAuth2 Login flow, the browser does **not** send Google's ID token with every request to `/profile`.

After the Google login completes:

```
Spring authenticates the user
            ↓
Authentication is stored in the SecurityContext
            ↓
The SecurityContext is associated with the HTTP session
            ↓
The browser sends the session cookie on later requests
```

This application is therefore session-based after login.

OAuth2 Login and OAuth2 Resource Server are different use cases:

Feature	Typical use
`oauth2Login()`	Log a browser user into the application and usually maintain a session
`oauth2ResourceServer().jwt()`	Protect an API that receives bearer access tokens

If a separate frontend or mobile application must call our API with bearer tokens, that requires a different architecture from this server-side session login demo.

22. Complete Flow Recap

```
1. User opens /oauth2/authorization/google
2. Spring redirects the browser to Google
3. Google authenticates the user
4. User approves the requested scopes
5. Google redirects back with an authorization code
6. Spring exchanges the code through the back channel
7. Spring validates and processes the OIDC response
8. Spring creates an OidcUser
9. CustomOidcUserService saves or updates AppUser
10. Spring stores the authenticated state in the session
11. The user accesses /profile as an authenticated user
```

Final Mental Model

```
Google password
→ stays with Google

Authorization code
→ short-lived value returned through the browser

Access token
→ used to access a protected resource

ID token
→ tells the client who authenticated

Refresh token
→ obtains new access tokens

provider + subject
→ links the Google identity to our internal user

session cookie
→ keeps the browser authenticated with our Spring application
```

> Google proves the external identity. Our application still owns its internal user record, roles and business rules.